

Briefing de Execução — Backend Scathon

Sep 26, 2026 · preparado por @Alfredo

1. Visão geral e stack

O backend passa a ser uma API Flask separada do frontend Next.js (que continua hospedado na Vercel) — Flask não roda bem como função serverless (conexão de banco persistente, migrations, jobs de webhook), então este projeto sobe num host próprio (Render, Railway, Fly.io ou um VPS) e o Next chama essa API por HTTP.

Camada	Biblioteca	Papel
Framework	Flask 3.x	Roteamento, app factory, blueprints
ORM	SQLAlchemy 2.x (via Flask-SQLAlchemy)	Modelos, queries parametrizadas (proteção nativa contra SQL injection), relacionamentos
Migrations	Flask-Migrate (Alembic por baixo)	Versiona o schema do banco a cada mudança de modelo, com rollback
Documentação da API	flask-smorest	Gera o Swagger/OpenAPI automaticamente a partir de schemas Marshmallow, com validação de entrada de graça
Senha	bcrypt	Hash com salt por senha, nunca texto puro nem hash reversível
Sessão/token	PyJWT (ou flask-jwt-extended)	Emite e verifica o token que autentica cada request
Rate limiting	flask-limiter	Trava tentativas de login e endpoints sensíveis por IP/conta
CORS	flask-cors	Só aceita requests do domínio do frontend, não de qualquer site
Banco	PostgreSQL em produção, SQLite em dev local	Fora do repositório, credencial nunca committada

O frontend já tem uma peça de segurança pronta que este backend deve respeitar, não substituir: `src/proxy.ts` no Next barra `/admin` no servidor, checando um cookie assinado que a própria rota `POST /api/auth/login` do Next emite (ver

`src/lib/session.ts`). Com o Flask entrando em cena, essa rota do Next para de validar e-mail/senha contra o arquivo mock `@/lib/accounts.ts` e passa a repassar a chamada pro Flask (`POST /auth/login` de verdade, senha comparada com bcrypt contra o banco) — o Next continua emitindo seu próprio cookie `httpOnly` pro `/admin` exatamente como hoje, só que o papel (`admin` / `customer`) nesse cookie passa a vir do banco real, não de duas contas fixas no código.

2. Modelagem de dados

As tabelas abaixo são o que hoje já existe como mock espalhado em `src/lib/*.ts` e `src/data/products.ts` no frontend — a modelagem só dá nome/relacionamento real ao que o Next já assume que existe.

Entidade (tabela)	Campos principais	Relacionamentos	Substitui hoje
User	id, email (único), password_hash, display_name, phone, avatar_url, role (admin/customer), created_at	1-N com Address, Order, WishListItem, Review	<code>@/lib/accounts.ts</code> (contas fixas)
Address	id, user_id, label, cep, street, number, complement, neighborhood, city, state	N-1 com User	<code>@/lib/addresses.ts</code>
Product	id, slug (único), title, description, price, compare_at_price, category, style_code, sizes, rating, review_count, is_new, is_best_seller, availability, cover_image, image_url, hover_image_url	1-N com ColorVariant, ProductImage, OrderItem, Review, WishListItem	<code>src/data/products.ts</code>
ColorVariant	id, product_id, name, hex, image_url	N-1 com Product	<code>Product.colors[]</code>
ProductImage	id, product_id, url, kind (specimen/gallery), position	N-1 com Product	<code>Product.images[]</code> / <code>specimenImages[]</code>
Order	id (código tipo SCT-100482), user_id, status (processando/a caminho/entregue/cancelado), placed_at, shipping_method, shipping_carrier, shipping_cost	N-1 com User, 1-N com OrderItem, 1-1 com Payment	<code>@/lib/orders.ts</code> + <code>@/lib/adminOrders.ts</code>
OrderItem	id, order_id, product_id, color, size, price (snapshot), quantity, image_url (snapshot)	N-1 com Order e Product	<code>Order.items[]</code>
Payment	id, order_id, payment_id (Mercado Pago), method (pix/credito/boleto), status (approved/pending/refunded/rejected), gross_amount, fee_amount, net_amount	1-1 com Order	<code>AdminOrder.payment</code> , <code>@/lib/mercadoPago.ts</code>
Review	id, product_id, user_id, order_item_id (opcional), rating, comment, created_at	N-1 com Product, User	<code>Product.reviews[]</code> (marketing) + <code>@/lib/orderReviews.ts</code> (pós-compra) — unificados numa só tabela
WishListItem	user_id, product_id, created_at (chave composta)	N-N entre User e Product	<code>WishlistContext</code> / <code>localStorage</code>
WaitlistEntry	id, product_id, email, size, created_at	N-1 com Product	<code>@/lib/waitlist.ts</code>

Entidade (tabela)	Campos principais	Relacionamentos	Substitui hoje
<code>InventoryLevel</code>	<code>product_id</code> (chave), <code>quantity</code>	1-1 com <code>Product</code> (MVP: por produto, não por variante)	<code>@/lib/inventory.ts</code>
<code>AdminAuditLog</code>	<code>id</code> , <code>admin_user_id</code> , <code>action</code> , <code>target</code> , <code>metadata</code> (JSON), <code>created_at</code>	N-1 com <code>User</code>	novo — não existe no mock, entra pela seção 3

Duas decisões que valem registrar: `InventoryLevel` fica por produto (não por combinação cor+tamanho) pra bater com o mock atual — ir mais fino é uma evolução de fase 2, não bloqueia o lançamento; e o carrinho (`CartItem`) continua vivendo no `localStorage` do navegador como hoje (`CartContext`) em vez de virar tabela — só os pedidos finalizados (`Order`) precisam de verdade estar no banco.

3. Autenticação e segurança

A autenticação vira uma via de duas camadas, cada uma com sua função: o Flask é quem sabe se a senha está certa; o Next é quem decide se um browser entra em `/admin`.

Fluxo de login. `POST /auth/login` no Flask recebe e-mail/senha, busca o `User` pelo e-mail e confere a senha com `bcrypt.checkpw` contra o `password_hash` guardado (nunca a senha em texto puro, nunca um hash reversível tipo MD5/SHA sem salt). Se bater, o Flask assina um JWT (PyJWT, `HS256` ou `RS256`) com `sub` (id do usuário), `role` e expiração curta (ex.: 1h de access token). A rota `/api/auth/login` do Next (já existe) passa a chamar esse endpoint em vez de checar `@/lib/accounts.ts`, e usa a resposta pra emitir o cookie `httpOnly` que `src/proxy.ts` já verifica — o navegador do cliente nunca vê o JWT do Flask, só o Next troca esse token com o backend.

Autorização (RBAC). Todo endpoint que mexe com dado de admin (pedidos de todo mundo, clientes, estoque) fica atrás de um decorator tipo `@admin_required` que confere `role == "admin"` a partir do JWT decodificado — nunca a partir de um campo que o próprio request manda (um `role` vindo do corpo da requisição é ignorado/sobrescrito sempre). O cadastro público (`POST /auth/register`) força `role = "customer"` no servidor, sem exceção — mesma regra que `registerAccount` já segue no frontend hoje, só que agora é impossível de burlar porque não existe campo `role` aceito nesse endpoint.

Superfície de ataque coberta:

- Senhas: `bcrypt` com custo (`rounds`) 12+, nunca menos — e nunca logadas, nunca devolvidas em nenhuma resposta da API.
- Banco: 100% via SQLAlchemy (ORM) — nenhuma query com f-string/concatenação de SQL cru em lugar nenhum do projeto, o que elimina SQL injection por construção.

- Entrada: todo payload de request valida contra um schema Marshmallow (o mesmo que alimenta o Swagger) antes de tocar o banco — tipo, tamanho, formato, nada de campo extra aceito por acidente.
- Rede: `flask-cors` só libera o domínio real do frontend (nunca `*`); tudo atrás de HTTPS; cookies do Next com `Secure + httpOnly + SameSite=Lax` (já assim hoje).
- Força bruta: `flask-limiter` trava `/auth/login` (ex.: 5 tentativas/minuto por IP+e-mail) e endpoints de escrita sensíveis.
- Webhooks (Mercado Pago, Melhor Envio): a assinatura/segredo do webhook é conferida antes de aceitar qualquer notificação de pagamento — sem isso, qualquer um poderia forjar um "pagamento aprovado" batendo no endpoint.
- Segredos: `SECRET_KEY` /chave do JWT, credenciais do Mercado Pago/Melhor Envio e a URL do banco vivem só em variáveis de ambiente do host (nunca no repositório) — mesmo princípio que `SESSION_SECRET` já segue no Next.
- Auditoria: toda ação de admin que muda dado (mudar status de pedido, ajustar estoque) grava uma linha em `AdminAuditLog` — se algo for alterado indevidamente, dá pra saber quem/quando.

O checklist objetivo de "pronto pra produção" fecha a seção 9.

4. API REST e documentação Swagger

Com `flask-smorest`, cada schema Marshmallow vira documentação Swagger automaticamente — o time não escreve o `openapi.json` à mão, ele nasce do próprio código e fica disponível em `/swagger-ui` (e o JSON cru em `/openapi.json`) assim que a rota existe. Toda rota vive sob `/api/v1/...` (versionada desde o início), erro sempre no formato `{"error": "mensagem", "code": "algo_snake_case"}`, e listas grandes (pedidos, clientes) usam paginação por `page / per_page` na query string.

Método	Rota	Área	Quem acessa
POST	<code>/api/v1/auth/register</code>	Auth	Público (sempre cria <code>role=customer</code>)
POST	<code>/api/v1/auth/login</code>	Auth	Público
POST	<code>/api/v1/auth/logout</code>	Auth	Logado
GET	<code>/api/v1/auth/me</code>	Auth	Logado
GET	<code>/api/v1/products</code>	Catálogo	Público
GET	<code>/api/v1/products/{slug}</code>	Catálogo	Público

Método	Rota	Área	Quem acessa
GET	/api/v1/categories	Catálogo	Público
GET	/api/v1/me/orders	Conta	Cliente (só os próprios)
GET, POST	/api/v1/me/addresses	Conta	Cliente
DELETE	/api/v1/me/addresses/{id}	Conta	Cliente (dono do endereço)
GET, POST, DELETE	/api/v1/me/wishlist	Conta	Cliente
POST	/api/v1/products/{slug}/reviews	Conta	Cliente (com compra confirmada)
POST	/api/v1/waitlist	Catálogo	Público (produto <code>coming_soon</code>)
POST	/api/v1/shipping/quote	Checkout	Público/Cliente
POST	/api/v1/checkout	Checkout	Cliente (cria <code>Order</code> + preferência Mercado Pago)
GET	/api/v1/orders/{id}	Checkout	Dono do pedido ou admin
GET	/api/v1/admin/overview	Admin	Admin
GET	/api/v1/admin/orders	Admin	Admin
PATCH	/api/v1/admin/orders/{id}/status	Admin	Admin
GET	/api/v1/admin/customers	Admin	Admin
GET, PATCH	/api/v1/admin/inventory	Admin	Admin
POST	/api/v1/admin/shipping/dispatch	Admin	Admin
GET	/api/v1/admin/integrations	Admin	Admin
POST	/api/v1/webhooks/mercado-pago	Webhook	Mercado Pago (assinatura verificada)
POST	/api/v1/webhooks/melhor-envio	Webhook	Melhor Envio (assinatura verificada)

Esse mapa é quase 1 pra 1 com as funções mock que já existem hoje (`getRecentOrders` , `getAdminOrders` , `getStockFor` , `calculateDispatch` , `notifyInterest` ...) — cada uma virou o corpo de um endpoint acima, então o frontend não precisa reinventar como pede os dados, só trocar o `import` por um `fetch` .

5. Migrations e ambiente

Fluxo do dia a dia: `flask db init` uma única vez no começo do projeto; daí em diante, toda mudança de modelo SQLAlchemy passa por `flask db migrate -m "descrição"` (gera o arquivo de migration em `migrations/versions/`) e `flask db upgrade` (aplica no banco). Regra que evita dor de cabeça: uma migration já aplicada em produção nunca é editada depois — uma correção vira uma migration nova. `flask db downgrade` existe pra desfazer a última, mas só em dev/staging.

Seed de dados: um script `seed.py` recria em SQL real exatamente o que hoje é mock no frontend — as duas contas de teste (`cliente@scathon.com` / `admin@scathon.com` , senha `senha123` , agora com hash bcrypt de verdade em vez de texto puro), os produtos de `src/data/products.ts` , os seis perfis de `CUSTOMER_PROFILES` e o estoque determinístico de `SEED_STOCK` . Isso mantém o ambiente de desenvolvimento reconhecível pra quem já testou o frontend mock.

Bancos por ambiente: SQLite local (`sqlite:///dev.db` , zero setup) em desenvolvimento; PostgreSQL gerenciado (Render/Railway/Neon/RDS) em produção — a URL de conexão vem sempre de `DATABASE_URL` , nunca hardcoded, pra trocar de um pro outro sem mudar código.

Variáveis de ambiente mínimas:

Variável	Para quê
<code>DATABASE_URL</code>	String de conexão do banco
<code>SECRET_KEY</code> / <code>JWT_SECRET_KEY</code>	Assina os tokens de sessão do Flask
<code>FRONTEND_ORIGIN</code>	Domínio liberado no CORS (<code>flask-cors</code>)
<code>MERCADO_PAGO_ACCESS_TOKEN</code> , <code>MERCADO_PAGO_PUBLIC_KEY</code> , <code>MERCADO_PAGO_WEBHOOK_SECRET</code>	Checkout Pro e validação do webhook
<code>MELHOR_ENVIO_CLIENT_ID</code> , <code>MELHOR_ENVIO_CLIENT_SECRET</code> , <code>MELHOR_ENVIO_TOKEN</code>	Cotação/etiqueta real de frete
<code>BCRYPT_LOG_ROUNDS</code>	Custo do hash de senha (12 é um piso razoável)

Todas ficam no `.env` do host (nunca commitadas) — mesmo princípio do `.env.example` que já existe no repositório do frontend pro `SESSION_SECRET` .

Backup: produção com backup automático diário do Postgres (a maioria dos hosts gerenciados já oferece isso pronto — só precisa ser ligado) e teste periódico de restauração, não só confiar que o backup existe.

6. Integrações externas

As duas integrações já estão desenhadas no frontend — `@/lib/mercadoPago.ts` e `@/lib/melhorEnvio.ts` têm mock funcionando e o comentário de cada função já mostra o `fetch` real que vai substituí-la. O trabalho aqui é mover essa chamada real pro Flask (nunca pro browser), porque as duas exigem credenciais secretas.

Mercado Pago (Checkout Pro): `POST /api/v1/checkout` cria o `Order + OrderItem`s no banco com status `processando`, chama a API do Mercado Pago pra criar uma "preferência" de pagamento com o `MERCADO_PAGO_ACCESS_TOKEN`, e devolve o `init_point` pro Next redirecionar o cliente. Quando o cliente paga, o Mercado Pago chama de volta `POST /api/v1/webhooks/mercado-pago` — esse endpoint confere a assinatura com `MERCADO_PAGO_WEBHOOK_SECRET` antes de aceitar qualquer coisa, busca o pagamento real na API do Mercado Pago (nunca confia só no payload do webhook) e atualiza o `Payment / Order` com o resultado. É esse webhook que faz `AdminOverviewTab` bater com o painel real do Mercado Pago, como o comentário de `adminOrders.ts` já previa.

Melhor Envio: `POST /api/v1/shipping/quote` (cliente, na sacola) e `POST /api/v1/admin/shipping/dispatch` (admin, cotação livre) chamam `POST /api/v2/me/shipment/calculate` da Melhor Envio com o `MELHOR_ENVIO_TOKEN`, exatamente como o comentário de `calculateDispatch` já esboça — só muda de onde a chamada sai (do servidor Flask, não do navegador). Gerar etiqueta de envio de verdade (comprar o frete) fica pra depois do MVP, quando o volume de pedidos justificar automatizar isso em vez de fazer manual.

Enquanto as credenciais reais não existem, os dois endpoints podem manter o mesmo mock determinístico de hoje rodando dentro do Flask (mesma fórmula, só em Python) — assim o resto do sistema (checkout, admin) já se integra com a rota certa e a troca pro provedor real é só trocar `connected: false` por `true` em `@/lib/integrations.ts` no dia em que as chaves chegarem.

7. Estrutura do projeto Python

App factory (`create_app()`) em vez de um `app = Flask(__name__)` solto no topo do arquivo — permite instanciar o app de um jeito pra testes (banco SQLite em memória) e de outro pra produção, e evita import circular entre extensões e blueprints. Uma pasta por área de negócio, espelhando o mapa de endpoints da seção 4:

```

scathon-backend/
  app/
    __init__.py      # create_app(), registra extensões e blueprints
    extensions.py    # db (SQLAlchemy), migrate, bcrypt, jwt, cors,
limiter
  models/           # um arquivo por entidade da seção 2
  schemas/          # Marshmallow - validação de entrada e forma da
resposta
  auth/             # blueprint: register, login, logout, me
  catalog/          # blueprint: products, categories
  account/          # blueprint: orders, addresses, wishlist, reviews
  checkout/         # blueprint: shipping quote, checkout, orders/{id}
  admin/            # blueprint: overview, orders, customers,
inventory, shipping
  webhooks/         # blueprint: mercado-pago, melhor-envio
  integrations/     # mercado_pago.py, melhor_envio.py - as chamadas
HTTP reais
  migrations/       # gerado pelo Flask-Migrate
  tests/            # um arquivo de teste por blueprint
  seed.py
  config.py         # lê as variáveis de ambiente da seção 5
  wsgi.py           # ponto de entrada em produção (gunicorn
wsgi:app)
  requirements.txt

```

Testes: `pytest` + `pytest-flask`, banco SQLite em memória por teste (fixture que recria o schema a cada teste, nunca compartilha estado entre eles) — no mínimo um teste por endpoint cobrindo o caminho feliz e o caso "sem permissão"/"não autenticado" (esse último é o que efetivamente comprova que o RBAC da seção 3 funciona, não só que existe no código).

Lint/formatação: `ruff` (lint + formatação, substitui flake8+black+isort num só binário) rodando no CI a cada push, mesma ideia do `pnpm lint` que já roda no frontend.

8. Plano de execução em fases

Cada fase só começa quando a anterior está rodando de ponta a ponta — nada de paralelizar autenticação com admin antes do básico estar de pé.

1. **Setup.** Repositório Flask com app factory, `config.py`, Postgres provisionado no host escolhido, CI rodando lint+testes a cada push, deploy mínimo ("hello world" respondendo em produção).

2. **Dados e autenticação.** Todos os modelos da seção 2, primeira migration, `seed.py`, `bcrypt+JWT`, endpoints de `/auth/*`. Critério de pronto: o Next chama `/auth/login` de verdade (não mais `@/lib/accounts.ts`) e o cookie de `/admin` reflete o papel vindo do banco.
3. **Catálogo.** Produtos/categorias migram de `src/data/products.ts` pro banco; endpoints públicos de produto. Critério de pronto: a home e as páginas de categoria do Next buscam da API, não mais do arquivo estático.
4. **Conta do cliente.** Endereços, wishlist, waitlist e avaliações via API. Critério de pronto: `/account`, `/wishlist` e o botão "avisar quando chegar" funcionam sem `localStorage`.
5. **Pedidos e checkout.** `Order / OrderItem / Payment` reais, endpoint de checkout, cotação de frete (mock determinístico dentro do Flask, sandbox do Mercado Pago). Critério de pronto: um pedido de teste completa o fluxo e aparece no banco com status certo.
6. **Admin.** Overview, pedidos, clientes, estoque e despacho de frete via API real. Critério de pronto: `<AdminView/>` no Next não importa mais nenhum arquivo de `@/lib/admin*`, só chama a API.
7. **Integrações reais e hardening.** Credenciais reais do Mercado Pago/Melhor Envio, webhooks com assinatura verificada, rate limiting e CORS travados pro domínio final, auditoria de admin ligada, checklist da seção 9 fechado. Critério de pronto: pode receber tráfego real sem ser o próprio time testando.

9. Checklist de segurança objetivo

Nada abaixo é opcional pra considerar o backend pronto pra receber tráfego real — é a lista que fecha o "não sermos invadidos" do pedido original.

- ☐ Senha com `bcrypt` (custo ≥ 12), nunca texto puro, nunca aparece em log nem em resposta da API
- ☐ Nenhuma query SQL crua/concatenada em lugar nenhum — só SQLAlchemy parametrizado
- ☐ Todo payload de entrada validado por schema (Marshmallow) antes de tocar o banco
- ☐ JWT com expiração curta, chave secreta forte (≥ 32 bytes aleatórios) fora do repositório
- ☐ RBAC checado a partir do token decodificado no servidor — nunca de um campo `role` enviado pelo cliente
- ☐ Cadastro público sempre força `role=customer`, sem exceção possível
- ☐ CORS restrito ao domínio real do frontend, nunca `*`
- ☐ HTTPS obrigatório em produção, cookies `Secure` + `httpOnly` + `SameSite`

- ☐ Rate limiting em `/auth/login` e em todo endpoint de escrita sensível
- ☐ Webhooks do Mercado Pago e da Melhor Envio validam assinatura antes de aceitar qualquer payload
- ☐ Segredos (chaves, tokens, URL do banco) só em variável de ambiente do host, nunca no repositório
- ☐ Log de auditoria pra ação de admin que muda dado (quem, quando, o quê)
- ☐ Dependências do `requirements.txt` com versão fixada e auditadas periodicamente (`pip-audit`)
- ☐ Backup automático do banco com teste de restauração real, não só assumido
- ☐ `DEBUG=False` em produção — erro nunca devolve stack trace/detalhe interno pro cliente
- ☐ Teste automatizado de "sem permissão"/"não autenticado" pra cada rota protegida, não só o caminho feliz
- ☐ Swagger deixa explícito quais rotas exigem token — documentação não é um convite pra testar sem autorização